Name :- Pratiksha Madhav Tonde

Domain :- AI-ML

Branch :- Computer Engg.

Clg name :- T.B.Girwalkr polytechnic ,Aambajogai

1)

```
pip install streamlit
```

```
Requirement already satisfied: streamlit in /usr/local/lib/python3.12/dist-packages (1.60.0)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.5.
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (8.4.2)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit) (3.1.51
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (26.2)
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (11.3.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.9.3)
Requirement already satisfied: protobuf<8,>=3.20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.29.6)
Requirement already satisfied: pyarrow<25,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (18.1.0)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (9.1.4)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (4.16.0
Requirement already satisfied: starlette<2,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1.3.1)
Requirement already satisfied: uvicorn<1,>=0.30.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.51.0)
Requirement already satisfied: httptools<1,>=0.6.3 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.8.0)
Requirement already satisfied: anyio<5,>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (4.14.2)
Requirement already satisfied: python-multipart<1,>=0.0.10 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.0.32)
Requirement already satisfied: websockets<17,>=12.0.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (15.0.1)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->streamli
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0-
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=4.0.0->streamlit) (3.18)
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7->s
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<4,>=1.4.0->stream
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<4,>=1.4.0->streamlit) (2025
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<4,>=1.4.0->streamlit) (20
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->str
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicorn<1,>=0.30.0->streamlit) (0.16.0
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython!=3.1
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->altair!=5.4.0,!=5.4.1,<
```

2)

```
import pandas as pd
import streamlit as st
```

```
from google.colab import files
uploaded=files.upload()
```

```
Choose Files bank.csv
bank.csv(text/csv) - 930123 bytes, last modified: 7/22/2026 - 100% done
Saving bank.csv to bank.csv
```

```
filepath = '/content/bank.csv'
df=pd.read_csv(filepath)
```

```
import os
print(os.listdir())
```

```
['.config', 'logs.txt', 'app.py', 'log.txt', 'bank.csv', 'sample_data']
```

```
import pandas as pd
import joblib
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing  import StandardScaler
from sklearn.linear_model import LinearRegression
```

3)

```
df.head()
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous
0	59	admin.	married	secondary	no	2343	yes	no	unknown	5	may	1042	1	-1	0
1	56	admin.	married	secondary	no	45	no	no	unknown	5	may	1467	1	-1	0
2	41	technician	married	secondary	no	1270	yes	no	unknown	5	may	1389	1	-1	0
3	55	services	married	secondary	no	2476	yes	no	unknown	5	may	579	1	-1	0
4	54	admin.	married	tertiary	no	184	no	no	unknown	5	may	673	2	-1	0

Next steps: Generate code with df | New interactive sheet

```
df_encoded = pd.get_dummies(df, columns=['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'pou
X = df_encoded.drop("balance",axis=1)
Y = df_encoded["balance"]
```

```
X_train,X_test,Y_train,Y_test = train_test_split(
    X,Y,
    test_size=0.2,
    random_state=42
)
```

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, Y_train)
print("Model Trained Successfully")
```

```
Model Trained Successfully
```

```
joblib.dump(model,"LR_Model.pkl")
joblib.dump(X.columns.tolist(),"columns.pkl")
print("All files save")
```

Terminal

4)

```
import os
print(os.listdir())
```

```
['.config', 'logs.txt', 'LR_Model.pkl', 'app.py', 'log.txt', 'columns.pkl', 'bank.csv', 'sample_data']
```

```
%%writefile app.py
import streamlit as st
import pandas as pd
import joblib
model = joblib.load("LR_model.pkl")
scaler = joblib.load("scaler.pkl")
encoded_columns = joblib.load("columns.pkl")
st.set_page_config(page_title="bank balance Predictor", layout="centered")
st.title("bank balance Predictor")
st.write("Enter the balance details .")
```

```
Overwriting app.py
```

```
!ls
```

```
app.py  bank.csv  columns.pkl  logs.txt  log.txt  LR_Model.pkl  sample_data
```

```
!find /content -name "app.py"
```

```
/content/app.py
```

```
%%writefile app.py
```

```
Overwriting app.py
```

```
!pip install streamlit pyngrok pandas joblib -q
```

```
from pyngrok import ngrok
ngrok.set_auth_token("3GezwGWGhipfz95MHZ0ENT7NDCF_5TQujZ7KYZeN7NPzasieM")
print("Auth Token Set Successfully")
```

5)

```
!streamlit run app.py
```

```
Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.

2026-07-22 16:41:48.420 Uvicorn server started on :::8502

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8502
  Network URL: http://172.28.0.12:8502
  External URL: http://35.196.39.100:8502

  Stopping...
```

```
!pip install streamlit pyngrok pandas joblib
```

```
Requirement already satisfied: streamlit in /usr/local/lib/python3.12/dist-packages (1.60.0)
Requirement already satisfied: pyngrok in /usr/local/lib/python3.12/dist-packages (8.1.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (8.4.2
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from st
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (26.2)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/python3.12/dist-packages (from streamlit) (
```

6)

```
!streamlit run app.py &>/content/logs.txt
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
import joblib
df = pd.get_dummies(df)
X = df.drop("balance", axis=1)
y = df["balance"]
encoded_columns = X.columns
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
model = LinearRegression()
model.fit(X_train_scaled, y_train)
joblib.dump(model, "LR_model.pkl")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(encoded_columns, "columns.pkl")
print("Model files created successfully")
```

```
Model files created successfully
```

```
%%writefile app.py
import streamlit as st
import pandas as pd
import joblib
model = joblib.load("LR_model.pkl")
scaler = joblib.load("scaler.pkl")
encoded_columns = joblib.load("columns.pkl")
st.set_page_config(page_title="Bank Balance Predictor", layout="centered")
```

7)

```
    user_input_encoded = user_input_encoded[encoded_columns]

    numerical_cols = ['age', 'day', 'duration', 'campaign', 'pdays', 'previous']
    user_input_scaled = user_input_encoded.copy()
    user_input_scaled[numerical_cols] = scaler.transform(user_input_scaled[numerical_cols])
    predicted_balance = model.predict(user_input_scaled)

    st.subheader("Predicted Bank Balance:")
    st.success(f"The predicted bank balance is: ${predicted_balance[0]:,.2f}")
```

```
Overwriting app.py
```

```
!streamlit run app.py &>/content/logs.txt
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
data = pd.DataFrame({
    'job': ['admin', 'blue-collar', 'technician', 'admin', 'retired'],
    'marital': ['married', 'single', 'married', 'divorced', 'married'],
    'education': ['secondary', 'primary', 'tertiary', 'secondary', 'primary'],
    'housing': ['yes', 'no', 'yes', 'no', 'yes'],
    'deposit': [1, 0, 1, 0, 1]  # Target variable (1 = yes, 0 = no)
})
X = data[['job', 'marital', 'education', 'housing']]
y = data['deposit']
encoder = OneHotEncoder(sparse_output=False, drop='first')
X_encoded = encoder.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Model Accuracy:", accuracy_score(y_test, y_pred))
```

```
Model Accuracy: 1.0
```

8)

```
    }

    df = pd.DataFrame(data)
    pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(stop_words='english')),
        ('classifier', LogisticRegression())
    ])
    pipeline.fit(df['feedback'], df['churn'])
    return pipeline
model = train_model()
st.title("🏦 Bank Customer Churn Predictor")
st.write("Analyze customer feedback to predict if they will leave the bank.")
user_feedback = st.text_input(
    label="Enter customer feedback or review:",
    placeholder="Type here (e.g., 'The staff was very rude and unhelpful')"
)
if st.button("Predict"):
    if user_feedback.strip() == "":
        st.warning("Please enter some text before predicting.")
    else:
        prediction = model.predict([user_feedback])[0]
        probabilities = model.predict_proba([user_feedback])[0]
        churn_chance = probabilities[1] * 100
        st.subheader("Result:")
        if prediction == 1:
            st.error(f"⚠️ High Risk of Churn! ({churn_chance:.1f}% probability)")
            st.write("The customer is likely to leave the bank based on this feedback.")
        else:
            st.success(f"✅ Low Risk of Churn. ({100 - churn_chance:.1f}% probability of staying)")
            st.write("The customer seems satisfied with the service.")
```

```
2026-07-22 16:57:28.930 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRun
2026-07-22 16:57:28.958 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRun
2026-07-22 16:57:29.022
  Warning: to view this Streamlit app on a browser, run it with the following
  command:

    streamlit run /usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py [ARGUMENTS]
2026-07-22 16:57:29.023 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

Terminal

9)

```
df = pd.DataFrame(data)
print("--- Original Bank DataFrame ---")
print(df)
print("\n" + "="*50 + "\n")
df_encoded = pd.get_dummies(df, columns=['job'], prefix='job', dtype=int)
edu_order = ['primary', 'secondary', 'tertiary']
ordinal_enc = OrdinalEncoder(categories=[edu_order])
df_encoded['education'] = ordinal_enc.fit_transform(df[['education']]).astype(int)
binary_map = {'yes': 1, 'no': 0}
df_encoded['housing_loan'] = df_encoded['housing_loan'].map(binary_map)
label_enc = LabelEncoder()
df_encoded['subscribed'] = label_enc.fit_transform(df_encoded['subscribed'])
print("--- Encoded DataFrame for Machine Learning ---")
print(df_encoded)
```

```
--- Original Bank DataFrame ---
   age          job  education housing_loan  balance subscribed
0   32   management   tertiary           no     2500        yes
1   45  blue-collar    primary          yes     -120         no
2   22   technician  secondary          yes      450         no
3   54   management   tertiary           no     8900        yes
4   39     services  secondary          yes     1500         no

==================================================

--- Encoded DataFrame for Machine Learning ---
   age  education  housing_loan  balance  subscribed  job_blue-collar  \
0   32          2             0     2500           1                0
1   45          0             1     -120           0                1
2   22          1             1      450           0                0
3   54          2             0     8900           1                0
4   39          1             1     1500           0                0

   job_management  job_services  job_technician
0               1             0               0
1               0             0               0
```

10)

```
df['housing'] = le.fit_transform(df['housing'])
df['loan'] = le.fit_transform(df['loan'])
df['y_subscribed'] = le.fit_transform(df['y_subscribed'])
X = df.drop(columns=['y_subscribed'])
y = df['y_subscribed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\n--- Scaled Feature matrix (First 2 rows) ---")
print(X_train_scaled[:2])
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print(f"\n--- Model Evaluation ---")
print(f"Prediction Accuracy: {accuracy * 100:.2f}%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
--- Original Bank Dataset Sample ---
   age          job  balance housing loan  duration y_subscribed
0   33   management     2143     yes   no       261           no
1   44  blue-collar       29     yes   no       151           no
2   28   management        2     yes   no        76           no

--- Scaled Feature matrix (First 2 rows) ---
[[-0.20069496  0.15249857  0.37555118  1.          0.          0.23293575]
 [-0.46828825 -1.06748999  1.97324895  1.          0.          0.7328954 ]]

--- Model Evaluation ---
Prediction Accuracy: 50.00%

Confusion Matrix:
[[1 1]
 [0 0]]
```

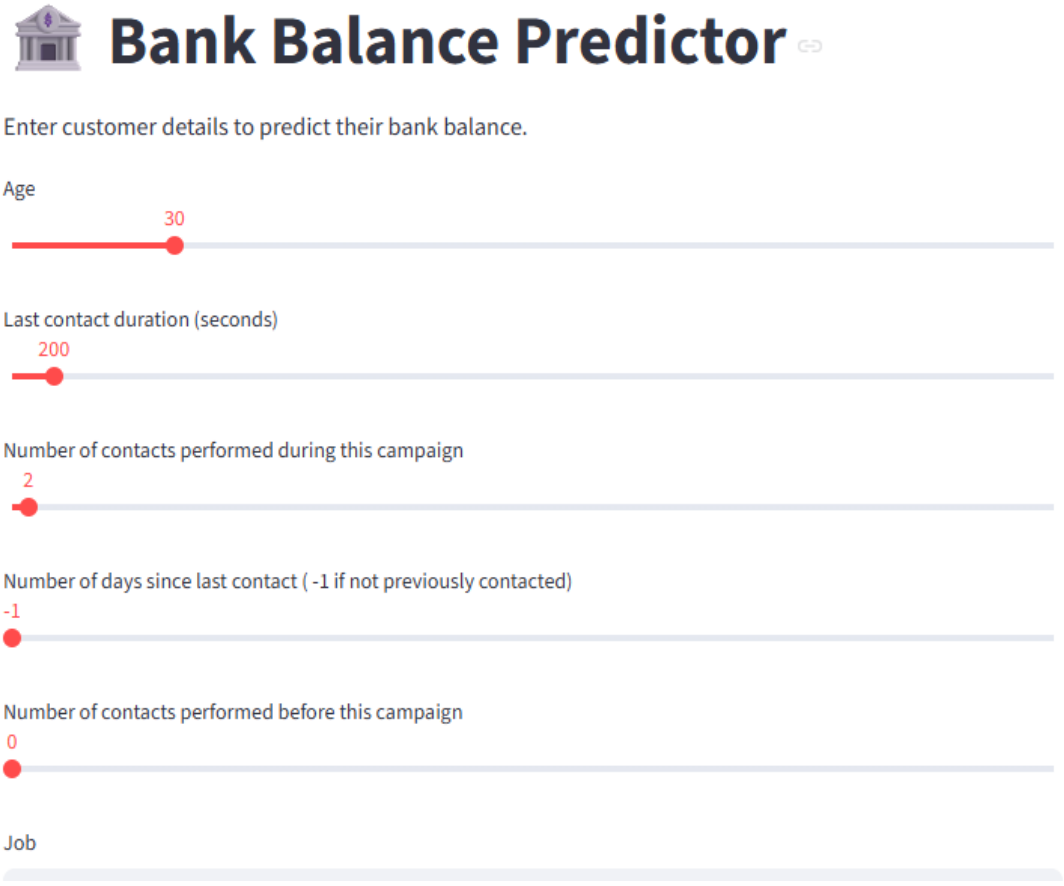
11)

```
for i in range(MAX_RETRIES):
    try:
        print(f"Attempt {i + 1}/{MAX_RETRIES}: Starting Streamlit app and ngrok tunnel...")
        !nohup streamlit run app.py &>/content/logs.txt &
        time.sleep(10)
        public_url = ngrok.connect(addr='8501')
        print("\nStreamlit App Link:", public_url)
        break
    except Exception as e:
        print(f"Attempt {i + 1} failed: {e}")
        if i < MAX_RETRIES - 1:
            print(f"Retrying in {RETRY_DELAY} seconds...")
            ngrok.kill()
            !kill $(lsof -t -i:8501) || true
            !pkill -f ngrok || true
            time.sleep(RETRY_DELAY)
            RETRY_DELAY *= 1.5
        else:
            print("All retry attempts failed. Please check the logs and your ngrok token.")
            if public_url is None:
                print("Could not establish ngrok tunnel.")
```

```
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
^C
Attempt 1/5: Starting Streamlit app and ngrok tunnel...

Streamlit App Link: NgrokTunnel: "https://impulse-gallantly-trunks.ngrok-free.dev" -> "http://localhost:8501"
```

12)


Bank Balance Predictor
Enter customer details to predict their bank balance.
Age
30
Last contact duration (seconds)
200
Number of contacts performed during this campaign
2
Number of days since last contact (-1 if not previously contacted)
-1
Number of contacts performed before this campaign
0
Job

Marital Status

married

Education

primary

Has credit in default?

no

Has housing loan?

no

Has personal loan?

no

Contact communication type

cellular

Last contact month of year

jan

Has personal loan?

no

Contact communication type

cellular

Last contact month of year

jan

Outcome of the previous marketing campaign

failure

Has subscribed to term deposit? (Target for other models, keep as 'yes' or 'no')

yes

Last contact day of month

15

Predict Balance